

**420-414-H26  
Infonuagique**

**TP3  
Déploiement d'une infrastructure multi-service  
sur AWS (Docker)**

**Prénom & Nom :**

**Matricule :**

**Signature :**

## Contexte et objectif

*Dans ce travail pratique, on vous demande de concevoir, configurer puis déployer une infrastructure infonuagique basée sur Docker et Docker Compose, puis la déployer sur une instance EC2 (AWS).*

*Vous mettrez en place une architecture multi-services, accessible via un reverse proxy Traefik, avec un domaine personnalisé (DuckDNS) et un certificat SSL/TLS (HTTPS).*

Ce TP a pour but de vous évaluer sur les compétences suivantes :

- Conception d'une architecture multi-conteneurs avec Docker Compose
- Configuration d'un reverse proxy avec Traefik
- Gestion de variables d'environnement pour une application distribuée
- Déploiement d'une infrastructure sur AWS (réseau + instance)
- Configuration un nom de domaine avec DuckDNS.
- Mise en place d'un chiffrement SSL/TLS (HTTPS)

## Description du travail

Vous devez déployer une infrastructure Docker multi-services comprenant :

### 1. Service obligatoire

[Portainer](#) – Interface de gestion et de monitoring de votre environnement Docker

### 2. Choix de services (minimum 2 autres)

Vous devez choisir **au moins UN service dans cette liste** :

- [Jellyfin](#) – Serveur multimédia (similaire à Plex), permettant de gérer des bibliothèques de films, séries et musiques.
- [NextCloud](#) – Logiciel libre d'hébergement de fichiers et une plateforme de travail collaboratif (de type OneDrive et Office365).

Ainsi qu'**au moins UN service dans cette liste** :

- [Paperless-ngx](#) – Service de gestion et d'archivage de documents.
- [Plex](#) – Le plus populaire des serveurs médias.
- *BlogVerse* – Application de partage d'articles déployée au TP2
- Une application full-stack de votre choix

Voici une liste de lien pour vous guider dans le déploiement des services :

- **Portainer / Jellyfin / Nextcloud** - Laboratoire 1 du chapitre *Proxy et Load Balancer*.
- **Paperless**
  - [Video](#)
  - [docker-compose.yaml](#)
- **Plex**
  - [Documentation de l'image Docker Hub](#)

## Contraintes techniques

Votre infrastructure doit respecter les contraintes suivantes :

- L'orchestration doit être faite avec *Docker Compose*
- Tous les services doivent être accessibles via *Traefik*. *Traefik* doit agir comme reverse proxy et gérer automatiquement le chiffrement SSL/TLS (HTTPS).
- Vous devez utiliser un domaine fourni par DuckDNS
- Chaque service doit être accessible via un sous-domaine (ex: portainer.mondomaine.duckdns.org)

## Partie I – Conception (local + GitHub)

Dans cette partie, vous devez créer un dépôt GitHub contenant :

- **Fichier `compose.yaml`** – Ce fichier doit décrire tous vos services (en incluant *Traefik*), définir les réseaux nécessaires, configurer les labels Traefik (routing) et gérer les volumes (si applicable).
- **Fichier `.env.example`** – Ce fichier sert de documentation des variables d'environnement. Il doit inclure les variables pour *Traefik* (email, domaine, etc.) ainsi que les variables spécifiques aux applications.

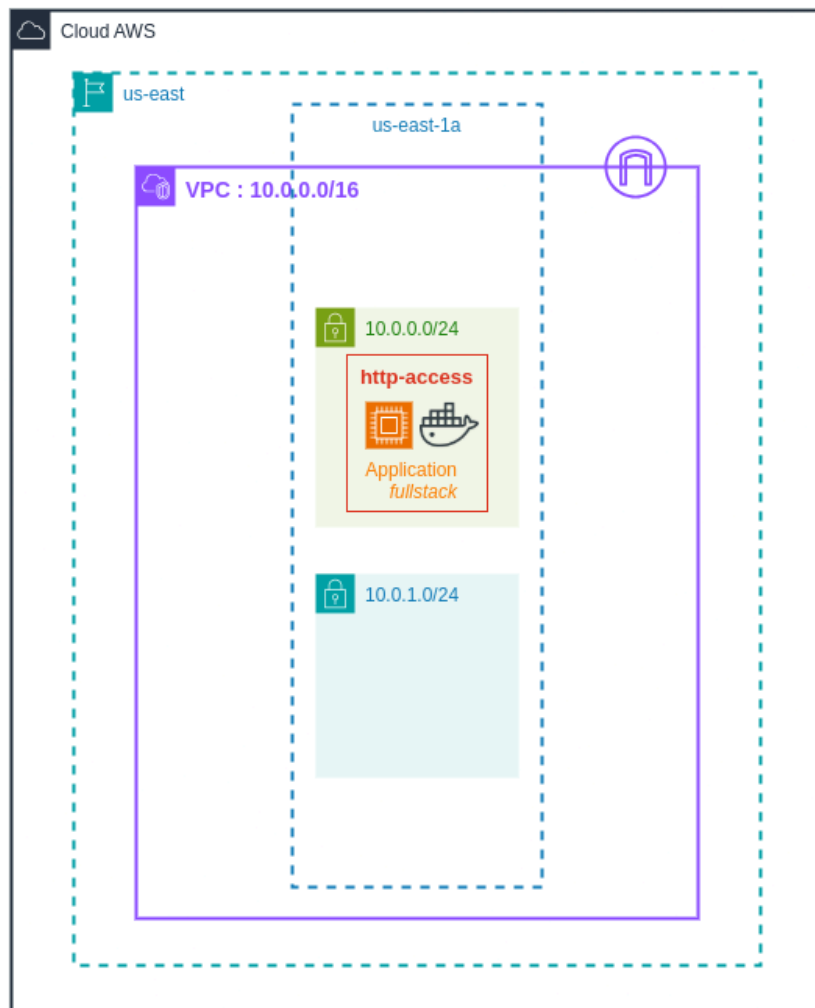
**Contrainte de nom** – Votre dépôt doit obligatoirement être nommé `tp3-<DA>`.

Structure attendue

```
tp3-<DA>/
| — compose.yaml
| — .env.example
| — README.md (optionnel mais recommandé)
```

## Partie II – Déploiement sur AWS

Vous devez recréer l'infrastructure suivante sur AWS :



### 1. Réseau (VPC)

**Nom** – tp3-<DA>-vpc

**Plage IPv4** – 10.<4e et 5e chiffre de votre DA>.0.0/16

**Région** – us-east-1

### 2. Sous-réseaux

#### 2.1. Sous-réseau public

**Nom** – tp3-<DA>-public-1

**Plage IPv4** – 10.<4e et 5e chiffre de votre DA>.0.0/24

**Type** – public

**Zone** – us-east-1a

#### 2.2. Sous-réseau privé

**Nom** – tp3-<DA>-private-1

**Plage IPv4** – 10.<4e et 5e chiffre de votre DA>.1.0/24

**Type** – privé  
**Zone** – us-east-1a

3. Tables de routage  
rtb-tp3-public (pour le sous-réseau public)  
rtb-tp3-private (pour le sous-réseau privé)
4. Passerelle Internet – une *Internet Gateway* attachée au VPC
5. Groupe de sécurité  
**Nom** – tp3-<DA>-sg  
**Autoriser** – SSH (port 22), HTTP (port 80), HTTPS (port 443) et d'autres ports si nécessaire.
6. Instance EC2  
**Nom** – tp3-<DA>-multi-service  
**AMI** – *Ubuntu 24.04 LTS*  
**Type** – t3.medium, t2.medium (minimum) ou t2.large (recommandé)  
**Sous-réseau** – public  
**Groupe de sécurité** – tp3-<DA>-sg
7. Déploiement  
Sur votre instance :
  - Installer *Docker* et *Docker Compose*
  - Cloner votre dépôt GitHub
  - Créer votre fichier .env
  - Lancer `docker compose up -d`

## Consignes de remise

Vous avez le choix entre deux modes de remise :

### Option 1 – Démonstration en classe

Vous effectuez une démonstration en direct durant la séance de cours ou lors des plages de disponibilité de l'enseignant.

Lors de cette démonstration, vous devrez :

- Présenter votre dépôt GitHub :
  - Montrer la structure du projet
  - Expliquer le contenu du fichier `compose.yaml`
- Expliquer les étapes de déploiement sur AWS.
- Faire une démonstration fonctionnelle :
  - Accès aux services via le navigateur
  - Vérification du bon fonctionnement des applications
  - Vérification du certificat HTTPS (signé par *Let's Encrypt*)
  - Accès via noms de domaine (*DuckDNS*)

## Option 2 — Remise sur Omnivox

Vous devez remettre un document (PDF et/ou .zip) contenant les éléments suivants.

### Partie I

- URL de votre dépôt GitHub
- Explication de votre fichier `compose.yaml` :
  - Description des services
  - Rôle de *Traefik*
  - Organisation générale (réseaux, volumes, variables d'environnement)

### Partie II

1. **VPC** – Capture d'écran de l'onglet "*Mappage des ressources*" du VPC
2. **Sous-réseaux** – Capture d'écran de l'onglet "*Détails*" pour chaque sous-réseau (tp3-<DA>-public-1 et tp3-<DA>-private-1)
3. **Groupe de sécurité** – Capture d'écran des règles entrantes et sortantes du groupe de sécurité
4. **Instance EC2**

**Méthode de création et configuration** – décrivez brièvement :

- La méthode utilisée pour configurer l'instance (installation de Docker, commandes utilisées pour lancer les services, si un script user-data a été utilisé)
- Si user-data : Inclure le script complet utilisé

**Détails de l'instance** – Capture d'écran de l'onglet "*Détails*" incluant obligatoirement:

- Adresse IP publique
- Adresse IP privée
- DNS public
- ID de l'AMI utilisée
- Type de l'instance

**Sécurité de l'instance** – Capture d'écran de l'onglet "*Sécurité*"

**Fonctionnement des services**

- Capture d'écran du résultat de la commande `docker ps`
- Capture d'écran du navigateur incluant :
  - Le nom de domaine
  - Le certificat HTTPS valide
  - Les 3 services fonctionnels, accessibles via des sous-domaines.

5. **Nom de domaine**

Capture d'écran de votre configuration **DuckDNS**

**L'adresse IP associée doit correspondre à l'adresse IP publique de votre instance Amazon EC2**

### Règles importantes

- **Les captures doivent être lisibles.**
- Toute incohérence entre les contraintes imposées entraînera une pénalité.
- Une remise incomplète sera pénalisée.

## Grille de correction

| Critère                                                                                                                                                                                                                                                    | Points    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>I - Fonctionnalité globale</b>                                                                                                                                                                                                                          | <b>35</b> |
| <b>1. Déploiement des services</b><br><i>Portainer fonctionnel et accessible (5 pts)</i><br><i>Un des service de la première liste fonctionnel et accessible (5 pts)</i><br><i>Un des services de la deuxième liste fonctionnel et accessible (15 pts)</i> | <b>25</b> |
| <b>2. Reverse proxy avec Traefik</b><br><i>Reverse proxy fonctionnel et correctement implémenté</i><br><i>Règles de redirection par sous-domaine fonctionnelle et cohérente, certificat SSL/TLS</i>                                                        | <b>10</b> |
| <b>II - Configuration technique</b>                                                                                                                                                                                                                        | <b>25</b> |
| <b>1. Configuration Docker Compose</b><br><i>Services bien définis, réseaux correctement utilisés, volumes pertinents, variables d'environnement bien gérées, labels traefik corrects et cohérents.</i>                                                    | <b>20</b> |
| <b>3. Domaine + HTTPS</b><br><i>Domaine DuckDNS fonctionnel</i><br><i>Certificat HTTPS valide (Let's Encrypt via Traefik)</i>                                                                                                                              | <b>5</b>  |
| <b>III - Infrastructure AWS</b>                                                                                                                                                                                                                            | <b>20</b> |
| <b>1. Infrastructure réseau</b><br><i>VPC, sous-réseaux, tables de routage et IGW correctement configurés</i>                                                                                                                                              | <b>8</b>  |
| <b>2. Sécurité</b><br><i>Groupe de sécurité correctement configuré (SSH, HTTP, HTTPS)</i>                                                                                                                                                                  | <b>6</b>  |
| <b>3. Instance EC2</b><br><i>Instance créée selon les spécifications, installation de Docker et Docker compose</i><br><i>Déploiement des services fonctionnels</i>                                                                                         | <b>6</b>  |
| <b>IV - Qualité du dépôt Github et documentation</b>                                                                                                                                                                                                       | <b>20</b> |
| <b>1. Dépôt GitHub</b><br><i>Structure claire, respect des consignes de nommage, fichiers présents</i>                                                                                                                                                     | <b>5</b>  |
| <b>2. Documentation (.env et explications)</b><br><i>.env.example complet, explications claires</i>                                                                                                                                                        | <b>15</b> |
| <b>Bonus (optionnel – jusqu'à + 10 points)</b>                                                                                                                                                                                                             |           |
| <i>Utilisation d'un script user-data automatisé</i><br><i>Implémentation de plus de trois services</i>                                                                                                                                                     |           |

|       |     |
|-------|-----|
| TOTAL | 100 |
|-------|-----|

### Pénalités

- **Remise incomplète** – jusqu'à -20 points
- **Non-respect des consignes** – jusqu'à -10 points
- **Incohérences majeures** – jusqu'à -15 points